

AI-Powered Document Intelligence System for Automated Extraction, Validation, Search, and Question Answering

A PROJECT REPORT
SUBMITTED IN PARTIAL FULFILMENT OF THE
REQUIREMENTS FOR THE DEGREE OF
Master of Technology (Online)

IN
Artificial Intelligence

BY
Student Name

SR No. SR Number



Faculty of Engineering
Indian Institute of Science
Bangalore - 560012 (INDIA)

Month, Year

Declaration of Originality

I, Student Name, with SR No. SR Number hereby declare that the material presented in the project report titled

AI-Powered Document Intelligence System for Automated Extraction, Validation, Search, and Question Answering

represents original work carried out by me at Organisation Name as part of the project credit requirements of the Master of Technology (Online) degree in Artificial Intelligence at the Indian Institute of Science, between Join Month, Year to End Month, Year.

With my signature, I certify that:

- I have not manipulated any of the data or results.
- I have not committed any plagiarism of intellectual property. I have clearly indicated and referenced the contributions of others.
- I have explicitly acknowledged all collaborative research and discussions.
- I have understood that any false claim will result in severe disciplinary action.
- I have understood that the work may be screened for any form of academic misconduct.

Date: _____

Student Signature:

In our capacities as internal project guide and faculty mentor of the above-mentioned work, we certify that the above statements are true to the best of our knowledge, and we have carried out due diligence to ensure the originality of the report.

Internal Guide Name: Project Guide Name

Internal Guide Signature:

Organisation: Organisation Name

Faculty Mentor Name: IISc Faculty Mentor Name

Faculty Mentor Signature:

Department: Faculty Mentor Department Name

© Student Name
Month, Year
All rights reserved

Acknowledgements

I express my sincere gratitude to my faculty mentor, project guide, and the Indian Institute of Science for providing the academic framework and guidance required to carry out this M.Tech project. Their feedback helped shape the project from a simple document upload utility into a complete document intelligence prototype with extraction, validation, search, and question-answering capabilities.

I also thank my peers, reviewers, and colleagues for their discussions on practical document processing problems, user expectations, and the need for reliable evidence-backed answers. These discussions influenced the design of the validation layer, confidence reporting, and Agentic RAG workflow.

Finally, I thank my family and friends for their encouragement throughout the project. Their support made it possible to sustain the implementation, experimentation, and report writing phases of this work.

Abstract

Organisations receive a large volume of semi-structured and unstructured documents such as invoices, contracts, reports, scanned forms, and supporting records. Manual processing of such documents is slow, error-prone, difficult to audit, and hard to scale when users need both structured field extraction and natural-language access to document contents. This project presents an AI-powered document intelligence system that automates document ingestion, text extraction, document classification, key-field extraction, validation, search, and question answering.

The implemented prototype, named Document Intelligence, uses a React and Vite frontend with a FastAPI backend. Uploaded documents are stored with metadata in SQLite and processed through a modular pipeline. The pipeline detects whether a PDF contains embedded text or requires scanned-document processing. Digital PDFs use direct text extraction, while scanned pages use OCR-based extraction through Tesseract and EasyOCR. A LayoutLMv3-based vision-language component is used for document-type classification and layout-aware feature extraction. A confidence-weighted fusion step combines OCR and layout signals, after which rule-based and LLM-assisted entity extraction identify fields such as dates, amounts, invoice numbers, purchase order numbers, parties, and organisation names. Business validation rules flag suspicious or low-confidence fields for review. The system also indexes document text for keyword search and semantic retrieval.

A second contribution of the project is an Agentic Retrieval-Augmented Generation workflow for document question answering. Instead of directly passing a question to a language model, the workflow plans the query intent, performs structured-field lookup when possible, retrieves candidate evidence, grades evidence, builds a grounded context, and generates an answer with evidence snippets and confidence. This makes answers more auditable and reduces unsupported responses in comparison with a direct black-box question-answering interface.

The prototype was evaluated using the benchmark and telemetry hooks implemented in the application. At the current prototype stage, measured metrics include a field extraction F1 score of 0.8, Search Precision@5 of 0.25, Search Mean Reciprocal Rank of 1.0, an estimated throughput of 29.77 documents per hour, and a review rate of 0.152. These results show that the system is suitable as a working proof of concept for intelligent document processing, while also identifying areas for future improvement such as larger benchmark datasets, model fine-tuning, stronger table extraction, production authentication, and cloud-scale deployment.

Contents

Section	Draft page
Acknowledgements	4
Abstract	5
1. Introduction	7
2. Literature Review	10
3. Methodology and Implementation	13
4. Experiments and Results	18
5. Conclusions and Future Work	21
Appendix A. API Endpoints	23
Appendix B. Prototype Configuration	24
Appendix C. Recommended Screenshots	25
References	26

Introduction

Digital transformation has increased the volume of electronic records handled by organisations, but many business processes still depend on documents that are not immediately machine-actionable. Invoices, contracts, purchase orders, reports, scanned forms, and supporting files may be available as PDFs or images, yet the information required by downstream workflows is often hidden inside free-form text, tables, inconsistent layouts, and scanned pages. Manual processing of such documents requires human effort for reading, extracting key fields, validating values, searching past records, and answering business questions. This approach becomes expensive and slow as the number of documents grows.

Traditional OCR systems convert document images into text, but text extraction alone is not enough for end-to-end document intelligence. A useful system must also classify document type, preserve useful layout information, extract structured fields, detect uncertain outputs, validate extracted values, support retrieval, and provide human-readable answers grounded in document evidence. Recent progress in transformer-based language models, document vision-language models, sentence embeddings, and retrieval-augmented generation makes it possible to build such systems as integrated applications.

Problem Statement

The problem addressed in this project is the design and implementation of a practical AI-powered document intelligence system that can ingest business documents, extract structured information, validate the extracted fields, support document search, and answer natural-language questions over uploaded documents. The system should be usable through a web interface, maintain processing history, expose backend APIs, and provide auditable evidence for question-answering outputs.

The project focuses on the following research and engineering question:

How can OCR, document layout understanding, rule-based validation, semantic retrieval, and language-model-based answering be integrated into a usable document intelligence pipeline for semi-structured business documents?

Motivation

Document-heavy workflows often suffer from three practical gaps. First, extraction systems may produce raw text but not structured fields. Second, extracted values may be wrong or incomplete, but users are not told which fields require review. Third, users may need to ask questions across documents, but conventional search interfaces only return files or snippets. The proposed system addresses these gaps by combining extraction, validation, search, and question answering in one workflow.

The project is motivated by real-world use cases such as invoice processing, contract review, compliance checks, document search, and operational knowledge discovery. For example, a user may upload a contract and ask for its effective date, parties, payment terms, or risky clauses. A reliable system should answer using evidence from the document, indicate confidence, and avoid inventing values that are not present.

Objectives

The main objectives of the project are:

1. To build a web-based document intelligence prototype with document upload, dashboard, document listing, search, and question-answering screens.
2. To implement a backend processing pipeline for text extraction, scanned-document OCR, document classification, entity extraction, validation, and indexing.
3. To combine OCR and layout-aware document analysis for improved handling of digital and scanned documents.
4. To store documents, extracted fields, processing logs, and status information in a persistent database.
5. To implement RAG and Agentic RAG workflows for grounded question answering over uploaded documents.
6. To evaluate the prototype using extraction, search, throughput, and review-rate metrics.

Scope of the Work

The system is implemented as a prototype rather than a production enterprise platform. It supports common document formats such as PDF, PNG, JPG, JPEG, TIFF, and DOCX, with a maximum file size configured in the backend. The backend uses SQLite for local persistence and stores uploaded files in a configurable storage directory. The frontend provides the main user interface, while FastAPI exposes REST endpoints for upload, document operations, validation, search, statistics, evaluation metrics, and question answering.

The current implementation emphasises practical integration. It uses direct PDF text extraction for digital PDFs, OCR for scanned pages, LayoutLMv3 for document classification and layout features, rule-based entity extraction, optional LLM-assisted extraction, validation rules, keyword and database-backed search, semantic chunk retrieval, and a local LLM-backed answer generator.

Contributions

The major contributions of this project are as follows:

1. A modular document processing pipeline that separates ingestion, extraction, classification, fusion, validation, indexing, and status tracking.
2. A confidence-aware extraction model that flags fields for review when confidence is low or validation rules fail.
3. A web application that exposes document upload, dashboard, document details, search, and Agentic RAG interfaces.
4. An Agentic RAG workflow that plans user questions, retrieves and grades evidence, and generates grounded answers with traceable evidence snippets.
5. An evaluation layer that reports implemented benchmark metrics such as extraction F1, search precision, search MRR, throughput, and review rate.

Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 reviews OCR, document AI, entity extraction, validation, search, and retrieval-augmented question answering. Chapter 3 describes the system architecture, backend pipeline, database model, and Agentic RAG workflow. Chapter 4 presents the prototype setup, evaluation metrics, and observed results. Chapter 5 concludes the report and discusses limitations and future work.

Literature Review

Intelligent Document Processing

Intelligent Document Processing (IDP) refers to the automated conversion of unstructured or semi-structured documents into useful structured data and searchable knowledge. Unlike basic file storage, an IDP system attempts to understand the content of a document, extract key fields, classify the document, and support downstream business actions. Typical IDP pipelines include ingestion, OCR, layout analysis, key-value extraction, table extraction, validation, indexing, and human review.

Business documents are challenging because their layouts are not fixed. Invoices may use different templates, contracts may have long clauses and nested sections, reports may mix paragraphs and tables, and scanned files may contain skew, noise, low resolution, or missing embedded text. Therefore, a robust system often needs both rule-based and model-based components.

Optical Character Recognition

OCR is the foundation of document digitisation. It converts document images into machine-readable text. Tesseract is a widely used open-source OCR engine whose architecture and practical use have been described by Smith [1]. Modern OCR workflows often combine character recognition with image preprocessing, page segmentation, word confidence scores, and bounding boxes. In this project, Tesseract is used to extract full text, word-level confidence, and bounding boxes from scanned pages.

EasyOCR provides another OCR option with ready-to-use recognition models for multiple scripts and languages [2]. The prototype uses EasyOCR as an alternate OCR source and compares its output with Tesseract. The longer or more complete text result can be selected as the primary OCR result, while the other result is retained as an alternate source.

Transformer-Based Language Models

Transformer models have changed the design of natural language systems. BERT introduced deep bidirectional contextual representations that can be fine-tuned for many downstream tasks, including question answering and sequence classification [3]. Transformer encoders are useful for document intelligence because document text often contains relationships between fields, labels, values, and surrounding context.

Sentence-BERT extends transformer models to produce sentence embeddings using Siamese and triplet network structures [4]. Such embeddings are useful for semantic retrieval because they allow text chunks and user queries to be compared in vector space. The project uses the all-MiniLM-L6-v2 sentence embedding model for retrieval indexing and relevant chunk lookup.

Layout-Aware Document Understanding

Plain text alone may lose important visual cues from documents. For example, an amount near a label such as "Total" has a different meaning from an amount in a paragraph. Layout-aware models combine textual, spatial, and visual information for document understanding tasks.

LayoutLMv3 is a multimodal transformer for Document AI that uses unified text and image masking objectives and word-patch alignment during pre-training [5]. It supports document understanding tasks such as form understanding, receipt understanding, document visual question answering, document image classification, and layout analysis. In this project, LayoutLMv3 is used as the vision-language component for document classification and layout feature extraction.

Information Extraction and Validation

Information extraction identifies structured fields such as invoice number, purchase order number, date, amount, email, vendor name, buyer name, contract parties, and document type. Extraction can be performed using regular expressions, rule-based patterns, sequence labelling models, LLMs, or hybrid methods. In a prototype setting, rule-based extraction is attractive because it is transparent, fast, and easy to debug. However, rules may miss fields expressed in unfamiliar wording, so LLM-assisted extraction can complement them.

Validation is necessary because extraction confidence alone does not guarantee correctness. A date may be in the future, an amount may be negative or unusually large, an email address may have invalid syntax, or an organisation name may be a generic word rather than a company. The system therefore applies validation rules after extraction and marks uncertain fields for human review.

Search and Retrieval

Document search allows users to find documents based on keywords, extracted fields, or natural language queries. Traditional search can match terms from the stored full text, while semantic search uses vector embeddings to retrieve conceptually related text chunks. Ranking metrics such as Precision@k and Mean Reciprocal Rank are commonly used to evaluate retrieval quality.

In this project, the search layer combines keyword search with database-backed fallback search. The RAG layer additionally indexes document chunks for semantic retrieval, making it possible to retrieve evidence for natural-language question answering.

Retrieval-Augmented Generation

Retrieval-Augmented Generation combines a parametric language model with non-parametric retrieved knowledge. Lewis et al. introduced RAG for knowledge-intensive NLP tasks, showing how retrieved passages can improve factuality and specificity in generation [6]. In document intelligence, RAG is useful because answers can be grounded in uploaded documents rather than relying only on the language model's internal knowledge.

A direct RAG pipeline retrieves passages and asks an LLM to answer using those passages. However, practical document question answering benefits from more structure. A field lookup question such as "What is the invoice number?" should be answered from structured extractions if available. A comparison question may require multiple related clauses. A risk-review question may need evidence about obligations, penalties, termination, or expiry. The Agentic RAG approach in this project addresses these needs by planning the query, retrieving evidence, grading it, and then generating an evidence-backed answer.

Web Application Architecture

The implemented prototype follows a common web application architecture. The frontend is built with React [7] and Vite [8], while the backend is implemented using FastAPI [9]. SQLAlchemy is used as the database ORM layer [10]. This architecture separates user interaction from backend processing and makes the system easier to test, extend, and deploy.

Summary

The literature shows that effective document intelligence requires more than OCR. It requires document layout understanding, structured extraction, confidence-aware validation, search, semantic retrieval, and grounded generation. The proposed system integrates these components into a working prototype and evaluates the outcome using both extraction and retrieval metrics.

Methodology and Implementation

System Overview

The proposed system is implemented as a full-stack document intelligence application. The frontend provides user-facing workflows for upload, dashboard monitoring, document review, search, and Agentic RAG. The backend exposes REST APIs and executes the document processing pipeline. The database stores uploaded document metadata, extracted fields, processing logs, chat history, and processing status.

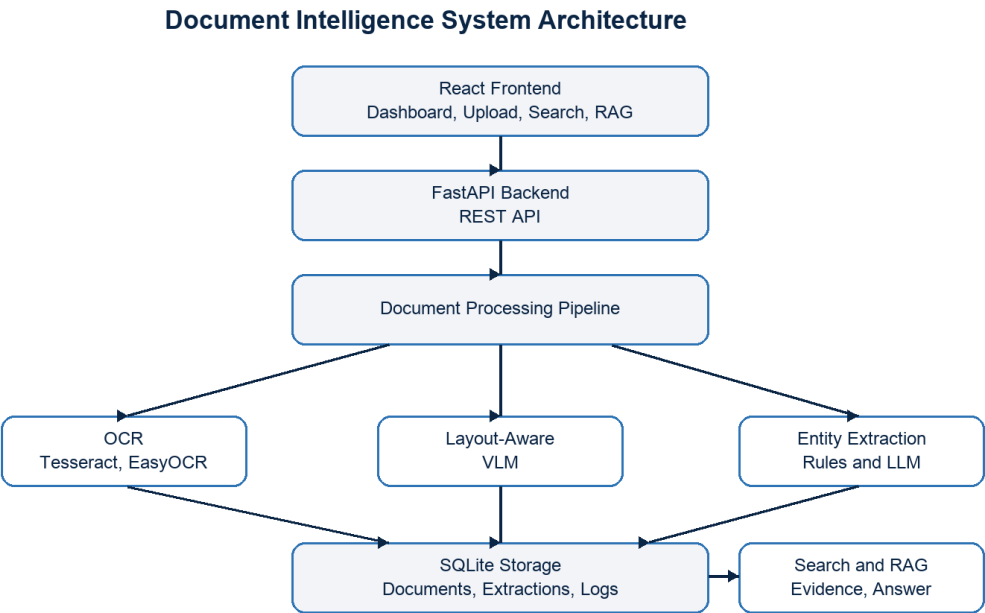


Figure 1: High-level architecture of the document intelligence system

Technology Stack

The frontend is built using React and Vite. The backend is implemented in Python using FastAPI. SQLAlchemy provides database access, and SQLite is used for local persistence. OCR is performed with Tesseract and EasyOCR. Layout-aware classification and feature extraction use LayoutLMv3. Semantic retrieval uses sentence embeddings, and the local language model backend is used for grounded answer generation.

Layer	Technology Used
Frontend	React, Vite, React Router
Backend API	FastAPI, Pydantic models

Database	SQLite, SQLAlchemy ORM
Text extraction	PyPDF2 for digital PDFs, OCR for scanned documents
OCR	Tesseract, EasyOCR
Document AI	LayoutLMv3
Retrieval	Keyword search, database fallback, semantic chunk retrieval
Question answering	Local LLM-backed RAG and Agentic RAG

Technology stack used in the prototype

Functional Modules

The application is organised into the following functional modules:

1. Document upload: accepts supported document formats and stores the original file with a unique identifier.
2. Processing pipeline: performs text detection, extraction, table extraction, text merge, entity extraction, boundary detection, validation, indexing, and status update.
3. Document management: lists uploaded documents, shows document details, extracted fields, and processing logs, and supports deletion or reprocessing.
4. Validation: applies field-specific validation rules and flags low-confidence or invalid values.
5. Search: retrieves documents using keyword and database-backed search.
6. Question answering: answers questions for a single document or across all documents using RAG.
7. Agentic RAG: performs query planning, structured lookup, evidence retrieval, evidence grading, and grounded answering.
8. Evaluation: exposes metrics for extraction, search, throughput, and review rate.

Database Design

The system uses four main database entities. The Document table stores file metadata, document type, processing status, page count, processing time, upload time, processed time, and full extracted text. The Extraction table stores field name, field value, confidence, source, page number, bounding box, and review flag. The ProcessingLog table records pipeline steps and messages for auditability. The ChatHistory table stores question-answer interactions.

The document status is represented using values such as uploaded, processing, completed, failed, and review needed. The document type is represented using values such as invoice, contract, form, report, and other. These enums make the UI and API responses easier to interpret.

Document Processing Pipeline

The backend pipeline starts when a user uploads a file. A unique document identifier is created, the file is stored, a database record is inserted, and a background task is scheduled for processing. The major steps are shown in Algorithm 1.

Document Processing Pipeline

1. Receive uploaded document and create database record
2. Mark document status as processing
3. Detect whether the PDF contains embedded text
4. If embedded text is available
5. Extract text directly from PDF pages
6. Classify the first page using the layout-aware model
7. Else
8. Convert pages to images
9. Run OCR and layout-aware processing on each page
10. Fuse OCR and layout results
11. End if
12. Extract table text where available
13. Merge page text and table text into full document text
14. Extract structured entities using rules and optional LLM extraction
15. Detect possible multi-document boundaries
16. Save extracted fields with confidence values
17. Validate fields using business rules
18. Index the document for keyword and semantic search
19. Mark the document as completed or review needed

Digital PDF and Scanned Document Handling

The pipeline uses two processing paths. For digital PDFs, embedded text is extracted directly. This is faster and avoids unnecessary OCR when reliable text is already present. For scanned documents, pages are converted to images and passed through OCR and layout-aware analysis. This design improves efficiency because digital PDFs and scanned files have different processing needs.

For scanned pages, Tesseract extracts text, word confidence, and bounding boxes. EasyOCR is used as an alternate recogniser. The OCR service compares the resulting text and selects the more informative output as the primary OCR result. The layout-aware service classifies document type and captures layout features. The fusion service combines OCR text with layout metadata and flags low-confidence words.

Entity Extraction

Entity extraction is performed over the merged document text. Rule-based extraction identifies common business fields such as dates, amounts, email addresses, vendor names, buyer names, parties, purchase order numbers, and invoice numbers. Extracted entities are normalised before storage. An optional LLM extraction step can contribute additional candidate fields not found by the rule-based extractor.

Each extracted field is stored with a confidence value and a source label. If the confidence value is below the configured threshold θ , the field is marked as requiring review. This allows the system to support human-in-the-loop verification instead of silently accepting uncertain outputs.

Validation Rules

Validation rules are applied after extraction. Date fields are checked for parseability and future dates. Amounts are checked for numeric format, non-negativity, and unusually high values. Email addresses are checked using a regular expression. Vendor and buyer names are checked against simple organisation-name heuristics and mock reference data. Purchase order numbers are checked against a mock list. Contract parties are checked for the presence of two plausible organisation names.

The validation stage does not discard uncertain fields. Instead, it updates the corresponding extraction record to indicate that review is needed. This design preserves extracted evidence while clearly informing the user that the value should be verified manually.

Search and Indexing

After extraction and validation, the document is indexed for search. The keyword search layer indexes full text and extracted fields. If in-memory keyword search does not return results, the database-backed search function is used as a fallback. For RAG, document text is split into chunks and indexed with embeddings. These chunks are later retrieved as evidence for question answering.

Agentic RAG Workflow

The Agentic RAG workflow improves over direct question answering by adding intermediate reasoning steps. First, the question is normalised and planned. The planner identifies intents such as field lookup, comparison, risk review, summary, or grounded QA. If the question is a structured field lookup and the extracted field exists, the answer is produced directly from the Extraction table. This is faster and more reliable for exact fields.

If structured lookup is not sufficient, the system retrieves candidate chunks using semantic retrieval. Candidate evidence is graded using a score that combines vector similarity, question-term coverage, and intent-specific bonuses. Only the top graded evidence is used to build the context for the answer generator. The final response contains the answer, intent, confidence, evidence snippets, trace steps, latency, and error status if applicable.

Agentic RAG Question Answering

1. Receive user question and optional document filter
2. Detect query intent and field target
3. If intent is field lookup and structured extraction exists
4. Return answer from extracted fields with confidence and evidence
5. Else
6. Retrieve candidate chunks using semantic search
7. Grade evidence using similarity, coverage, and intent bonuses
8. If best evidence score is below threshold
9. Return insufficient-evidence response

10. Else
11. Build grounded context from selected evidence
12. Generate answer using only selected evidence
13. Return answer, confidence, evidence, and trace steps
14. End if
15. End if

API Design

The backend exposes REST APIs under the prefix /api/v1. The major endpoints include document upload, document listing, document detail, deletion, reprocessing, validation, document-specific question answering, all-document question answering, agent-based answering, Agentic RAG answering, search, statistics, and evaluation metrics. These endpoints allow the frontend and external clients to use the same processing backend.

User Interface

The frontend contains pages for Dashboard, Upload, Documents, Document Detail, Search, and Agentic RAG. The dashboard displays processing statistics. The upload page lets users submit documents. The document pages allow review of extracted values and processing logs. The search page supports document retrieval. The Agentic RAG page allows users to ask questions and inspect evidence-backed answers.

Deployment Model

The application can run locally as a development prototype. The production-style path serves the built React frontend from the FastAPI backend, making the backend a single service for both APIs and the single-page application. This design simplifies local demonstration and can also be adapted for cloud deployment using a container image.

Experiments and Results

Evaluation Objectives

The evaluation of the prototype focuses on whether the implemented system can process uploaded documents, extract useful structured fields, support document retrieval, and expose measurable system behaviour. Since the project is a working prototype, the evaluation includes both benchmark metrics and operational telemetry. The primary objective is not to claim production-level accuracy, but to demonstrate that the integrated pipeline works and to identify measurable improvement areas.

Experimental Setup

The prototype was evaluated on the local application stack. The FastAPI backend served both the API and the built React single-page application. Documents were stored in the local storage directory and metadata was persisted in SQLite. The evaluation endpoint used the benchmark file configured under the backend evaluation storage path.

The live prototype snapshot used for this report contained five uploaded documents. Of these, two documents had completed processing and two required manual review. The average recorded processing time reported by the application statistics endpoint was 120941 ms, or approximately 120.94 seconds. This value includes the full processing path, including extraction, validation, indexing, and model-related overhead.

Metrics

The prototype exposes the following implemented metrics:

- 1. Field Extraction F1: measures the balance between precision and recall for key-value extraction against benchmark fields.
- 2. Search Precision@5: measures how often the top five search results contain relevant benchmark documents.
- 3. Search Mean Reciprocal Rank: measures how early the first relevant document appears in ranked search results.
- 4. Throughput: estimates the number of documents processed per hour based on recorded processing times.
- 5. Review Rate: measures the fraction of extracted fields flagged for manual review.

Question-answering accuracy is present as a planned metric in the evaluation endpoint, but it is not yet measured because a labelled QA benchmark has not been finalised. This is explicitly reported as pending rather than being treated as an achieved metric.

Results

The benchmark and telemetry results reported by the current prototype are shown in Table 2.

Metric	Measured Value	Status
--------	----------------	--------

Field Extraction F1	0.8	Measured
Search Precision@5	0.25	Measured
Search MRR	1.0	Measured
QA Answer Accuracy	Not measured	Planned
Throughput	29.77 docs/hr	Measured
Review Rate	0.152	Measured

Prototype evaluation metrics

The Field Extraction F1 score of 0.8 indicates that the prototype is able to extract benchmarked key fields with useful accuracy, while still leaving room for improvement. The Search MRR score of 1.0 indicates that, for the available benchmark search cases, the first relevant result appeared at rank one. The Search Precision@5 value of 0.25 indicates that while the top result may be good, the broader top-five ranking still contains non-relevant documents and should be improved.

The throughput value of 29.77 documents per hour is acceptable for a local proof of concept but is not yet optimised for production. The review rate of 0.152 shows that a measurable fraction of extracted fields were flagged for manual review. This is desirable from a safety perspective because the system should not silently accept uncertain extractions. However, a high review rate in production would increase user workload, so future improvements should reduce false review flags.

Operational Verification

In addition to benchmark metrics, the prototype was verified through live API checks. The health endpoint returned a healthy status. The root route served the frontend. The statistics endpoint returned document counts and processing time. The Agentic RAG route served the question-answering interface. These checks confirm that the application can be run as a single backend-served web application.

Route	Purpose
/health	Backend health check
/	Frontend single-page application
/api/v1/stats	Processing statistics
/api/v1/evaluation/metrics	Evaluation metrics
/agentic-rag	Agentic RAG user interface route

Operational routes used for prototype verification

Discussion

The results show that the prototype successfully integrates multiple document intelligence capabilities into one application. The extraction pipeline can process documents, store extracted fields, validate values, and index content. The search and retrieval layers allow documents to be discovered after processing. The Agentic RAG workflow adds a higher-level question-answering interface with traceable evidence.

The strongest current result is the Search MRR score, which indicates that relevant documents can appear early in ranked results for the benchmark cases. The Field Extraction F1 score is also promising for a prototype. The lower Precision@5 score suggests that search ranking should be improved using stronger semantic scoring, better chunking, metadata-aware ranking, or hybrid lexical-vector retrieval.

Processing time is influenced by document type. Digital PDFs can be processed faster because embedded text can be extracted directly. Scanned documents require OCR and layout-aware processing, which increases latency. Model loading and local LLM inference also contribute to runtime. Production deployment would benefit from model warm-up, asynchronous workers, GPU acceleration, batch processing, and persistent vector indexes.

Limitations of the Evaluation

The current evaluation has three important limitations. First, the benchmark dataset is small and should be expanded to include more invoices, contracts, forms, reports, scanned files, and mixed-quality documents. Second, QA accuracy has not yet been measured against labelled question-answer pairs. Third, the prototype is evaluated in a local environment, so cloud scalability, concurrent users, authentication, and enterprise integration are not fully tested.

Summary

The evaluation confirms that the implemented system is a functional proof of concept for AI-powered document intelligence. It demonstrates measurable extraction, search, throughput, and review behaviour. The next stage should expand the benchmark dataset, improve ranking quality, add QA ground truth, and evaluate the system under more realistic production conditions.

Conclusions and Future Work

Conclusion

This project designed and implemented an AI-powered document intelligence system for automated extraction, validation, search, and question answering. The system addresses the practical problem of converting semi-structured and unstructured business documents into searchable, validated, and queryable knowledge. It provides a web-based interface for users and a FastAPI backend that executes the core processing pipeline.

The implemented prototype supports document upload, digital PDF text extraction, scanned-document OCR, layout-aware document classification, entity extraction, validation, keyword search, semantic retrieval, and RAG-based question answering. The database stores document metadata, full text, extracted fields, processing status, processing logs, and question-answer history. The system therefore supports both automation and auditability.

A key contribution of this project is the Agentic RAG workflow. Instead of using a direct prompt-only approach, the workflow plans the question, attempts structured-field lookup, retrieves candidate evidence, grades evidence, builds a grounded context, and produces an answer with confidence and supporting snippets. This design is especially useful for document intelligence because users need answers that can be traced back to uploaded documents.

The prototype was evaluated using implemented benchmark and telemetry metrics. It achieved a Field Extraction F1 score of 0.8, Search Precision@5 of 0.25, Search MRR of 1.0, estimated throughput of 29.77 documents per hour, and review rate of 0.152. These values demonstrate that the prototype is functional and measurable, while also showing the need for broader datasets and improved ranking.

Key Learnings

The project produced several important learnings:

1. Digital PDFs and scanned documents require different processing paths. Direct text extraction is preferable for digital PDFs, while scanned documents require OCR and layout-aware handling.
2. OCR output alone is insufficient for reliable document intelligence. Confidence values, validation rules, and review flags are necessary to support trustworthy workflows.
3. Structured-field lookup is faster and more reliable than LLM generation for exact field questions.
4. Retrieval quality strongly affects answer quality in RAG systems. Evidence ranking must be evaluated and improved independently from answer generation.
5. A working UI is important for evaluating document intelligence because users need to inspect extracted values, logs, evidence, and review flags.

Limitations

The current system is a prototype and has several limitations. The benchmark dataset is small. QA accuracy has not yet been measured using labelled ground-truth answers. Table extraction is present but can be improved for complex table structures. The LayoutLMv3 component is used for classification and layout features, but the model has not yet been fine-tuned on a project-specific dataset. The application

currently uses local SQLite storage and local model execution, which are suitable for development but not sufficient for high-concurrency enterprise deployment.

The current validation layer uses a mixture of deterministic rules, heuristics, and mock reference data. A production deployment would require integration with real enterprise systems such as ERP, CRM, vendor master, contract repository, identity provider, and audit logging services.

Future Work

Future work can proceed in the following directions:

1. Dataset expansion: collect a larger labelled dataset of invoices, contracts, forms, reports, and scanned documents.
2. Model fine-tuning: fine-tune LayoutLMv3 or related models for document type classification, key-value extraction, and layout understanding on project-specific data.
3. Improved table extraction: add stronger table detection and structure reconstruction for financial and operational documents.
4. Hybrid retrieval: combine lexical ranking, metadata filters, vector search, and reranking to improve Precision@5.
5. QA benchmarking: create labelled question-answer pairs and measure exact match, semantic similarity, answer faithfulness, and citation correctness.
6. Human feedback loop: allow users to correct extracted fields and use those corrections to improve future extraction.
7. Production deployment: add authentication, role-based access, cloud object storage, queue-based workers, persistent vector database, monitoring, and autoscaling.
8. Security and governance: add document access controls, audit logs, data retention policies, encryption, and privacy-preserving model execution.

Final Remarks

The project demonstrates that OCR, document AI, validation, search, and retrieval-augmented question answering can be integrated into a practical full-stack system. While further work is required for production readiness, the prototype provides a strong foundation for an M.Tech project because it combines applied AI methods, software engineering, evaluation, and a working end-user application.

API Endpoints

Method	Endpoint	Description
POST	/api/v1/documents/upload	Upload document for processing
GET	/api/v1/documents	List uploaded documents
GET	/api/v1/documents/{id}	Fetch document details and extractions
DELETE	/api/v1/documents/{id}	Delete document and related records
POST	/api/v1/documents/{id}/reprocess	Re-run document processing
GET	/api/v1/documents/{id}/validate	Validate extracted fields
POST	/api/v1/documents/{id}/ask	Ask a question about one document
POST	/api/v1/ask	Ask a question across all documents
POST	/api/v1/agent/ask	Ask through backend agent router
POST	/api/v1/agent-rag/ask	Ask using Agentic RAG workflow
POST	/api/v1/search	Search documents
GET	/api/v1/stats	Get processing statistics
GET	/api/v1/evaluation/metrics	Get evaluation metrics

Major backend API endpoints

Prototype Configuration

The prototype uses the following important configuration values:

1. API prefix: /api/v1
2. Local database: sqlite:///./doc_intelligence.db
3. Supported formats: PDF, PNG, JPG, JPEG, TIFF, and DOCX
4. Maximum file size: 50 MB
5. VLM model: microsoft/layoutlmv3-base
6. Embedding model: all-MiniLM-L6-v2
7. Confidence threshold: 0.7

Recommended Screenshots for Final Submission

The final submitted report should include screenshots of the following pages after personal details and final data are confirmed:

1. Dashboard page showing document counts and processing summary.
2. Upload page showing the document upload workflow.
3. Document detail page showing extracted fields and processing logs.
4. Search page showing document retrieval results.
5. Agentic RAG page showing answer, evidence snippets, confidence, and trace steps.

References

1. Smith, R. (2007). An Overview of the Tesseract OCR Engine. Proceedings of the Ninth International Conference on Document Analysis and Recognition, 629-633. doi:10.1109/ICDAR.2007.4376991.
2. JaidedAI. (2026). EasyOCR: Ready-to-use OCR with 80+ supported languages. <https://github.com/JaidedAI/EasyOCR>. Accessed: 2026-06-08.
3. Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. arXiv:1810.04805.
4. Reimers, N., and Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. Proceedings of EMNLP 2019, 3982-3992. doi:10.18653/v1/D19-1410.
5. Huang, Y., Lv, T., Cui, L., Lu, Y., and Wei, F. (2022). LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking. arXiv:2204.08387.
6. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems, 33, 9459-9474.
7. React Team. (2026). React Documentation. <https://react.dev/>. Accessed: 2026-06-08.
8. Vite. (2026). Vite Documentation. <https://vite.dev/guide/>. Accessed: 2026-06-08.
9. FastAPI. (2026). FastAPI Documentation. <https://fastapi.tiangolo.com/>. Accessed: 2026-06-08.
10. SQLAlchemy. (2026). SQLAlchemy Documentation. <https://docs.sqlalchemy.org/>. Accessed: 2026-06-08.